SmartPlanif — Cœur métier de l'allocation

Pseudo-code de l'allocation d'un **bus**, d'un **chauffeur** et d'un **receveur** à chaque voyage d'une journée. Seul le noyau métier est décrit (Domain + orchestration) : ni accès base de données, ni interface.

1. Principe

```
ENTRÉES  : voyages à planifier, voyages verrouillés (déjà affectés), bus,
agents, indisponibilités, politique
SORTIE   : pour chaque voyage -> (bus, chauffeur, receveur, score,
justification)  ou  échec expliqué

ALLOUER(journée) :
   1. verrouiller les voyages déjà affectés (ils occupent leurs ressources)
   2. diagnostiquer la faisabilité (demande maximale vs ressources)
   3. construire un plan initial          (glouton OU par phases)
   4. l'améliorer                         (ALNS + recuit simulé, optionnel)
   5. compléter les voyages restants      (passe déterministe)
   6. calculer les scores exacts, puis contrôler le plan de façon
indépendante
   7. n'enregistrer que si les garde-fous sont satisfaits
```

2. Modèle métier

```
Agent  (Id, Rôle, JourDeRepos?, Centre?, Délégation?)   Rôle = Chauffeur |
Receveur | Polyvalent | Autre
Bus    (Id, Actif, Places, Catégorie?, Centre?, Délégation?, MiseEnService?)
Voyage (Id, Ligne, TypeDeLigne?, Départ, Arrivée, Km, Centre?, Délégation?,
StationDépart?, StationArrivée?)
         Intervalle = [Départ, Arrivée[        // bout à bout = pas de
chevauchement

Planning d'une ressource = liste triée de voyages
   Chevauche(intervalle), Précédent(t), Suivant(t), HeuresDuJour(j),
NbVoyagesDuJour(j), KmDuJour(j), Amplitude(j)

Carnet de plannings : un planning par bus ET UN SEUL planning par agent
   -> un agent ne peut jamais être chauffeur et receveur en même temps
   Inscrire(voyage, bus, chauffeur, receveur) / Retirer(...) / Copier()

État du plan = (voyages à planifier, carnet, affectations{voyage -> triplet})
   Affecter(a)  : inscrit au carnet
   Désaffecter(id) : retire du carnet, libère les ressources
   Copier()     : copie profonde
```

3. Contraintes : dures et souples

Chaque contrainte du catalogue est **Désactivée**, **Dure** ou **Souple (poids)**. Les impossibilités physiques (chevauchement, bus inactif) sont verrouillées en dures.

```
RÈGLE DURE  : Vérifier(ressource, voyage, planning, politique) -> violation |
rien      // un rejet est définitif
RÈGLE SOUPLE: Évaluer(...) -> valeur dans [0,1]  (1 idéal, 0 pire, 0,5
inconnu)         // points = poids x valeur

ConstruireRègles(définitions, politique) :
   POUR chaque définition, dans l'ordre (les plus économiques d'abord)
      mode = choix de la politique (sinon valeur par défaut)
      SI mode = Dure   ALORS ajouter la règle dure
      SI mode = Souple ET poids > 0 ALORS ajouter la règle souple pondérée
```

Règles dures :

```
Bus     B02 actif | B03 pas de chevauchement | B10 maintenance | B09 remise
en service >= 15 min
        B05 capacité >= passagers (si connu) | B11 km du jour <= 500 | B06
catégorie autorisée pour le type de ligne
Agent   A05 pas de chevauchement | A03 jour de repos | A04 congés | A07 repos
>= 30 min avant ET après
        A08 heures du jour + durée <= 8 h | A09 voyages du jour < 6 | A10
amplitude <= 12 h
Centre / délégation (B07, B08, A11, A12) : dure ou souple ; une valeur
inconnue n'élimine jamais
```

Règles souples (valeur dans [0,1]) :

```
Centre/délégation identiques   : 1 si égal, 0 sinon, 0,5 si inconnu
Peu de voyages (bus, agent)    : 1 / (1 + nbVoyagesDuJour)
Catégorie préférée (BP03)      : 1 préférée, 0,5 autorisée, 0 sinon
Capacité adaptée (BP04)        : passagers / capacité
Continuité de station (BP05)   : 1 si le voyage précédent finit où celui-ci
commence
Peu de km (BP06)               : 1 - km / 500
Bus récent (BP08)              : 1 - âge / 20 ans
Repos élevé (AP03)             : (départ - fin du précédent) / 4 h, plafonné
à 1
Peu d'heures (AP04)            : 1 - heuresDuJour / 8 h
Continuité de ligne (AP06)     : 1 si même ligne que le voyage précédent
```

4. Décision pour UN voyage

```
Sélectionner(candidats, voyage, règles, bruit?) :
   POUR chaque candidat
      SI une règle dure est violée : compter le rejet par code de règle ;
passer au suivant
      score = somme des (poids x valeur) des règles souples
      clé   = score + bruit aléatoire proportionnel à la somme des poids
// bruit seulement dans l'ALNS
   TRIER par clé décroissante, puis par Id croissant (déterminisme)

EssayerAffecter(voyage, ressources, carnet) :
   bus        = Sélectionner(tous les bus).meilleur
   chauffeurs = Sélectionner(agents aptes à conduire)
   receveurs  = (tous les agents polyvalents) ? chauffeurs :
Sélectionner(agents aptes receveur)
   chauffeur  = chauffeurs.meilleur
   receveur   = premier de receveurs DIFFÉRENT du chauffeur
```

```
   SI bus OU chauffeur OU receveur manque
        RETOURNER échec (rôles manquants + rejets cumulés par code)        //
tout ou rien
   RETOURNER affectation (score = bus + chauffeur + receveur)
   // ne modifie JAMAIS le carnet : l'appelant fait Affecter

Difficulté(voyage) :                          // petit = très contraint
   D = chauffeurs éligibles ; R = receveurs éligibles ; B = bus éligibles
   RETOURNER min(|B|, |D|, |R|, |D ∪ R| / 2)      // il faut deux personnes
distinctes
```

5. Faisabilité et construction du plan initial

```
PicSimultané(voyages, écart) :
   événements = (départ, +1) et (arrivée + écart, -1), triés par temps (les
-1 d'abord à égalité)
   RETOURNER le maximum du cumul

Diagnostic(voyages, ressources) :                     // phase 0
   bus      : PicSimultané(tous, remise en service)      vs bus actifs
   par type : PicSimultané(type)                          vs bus compatibles
avec ce type (si B06 dure)
   agents   : 2 x PicSimultané(tous, repos)               vs agents
disponibles (hors jour de repos)
   heures   : 2 x total des heures                        vs agents x 8 h
   chaque ligne : « Suffisant » si demande <= offre, sinon « INSUFFISANT »

ConstructionGloutonne :                               // référence de
comparaison
   POUR chaque voyage trié par (départ, Id) : EssayerAffecter ; SI succès
ALORS Affecter

ConstructionParPhases :
   POUR chaque type de ligne : tension = PicSimultané(type) / max(1, bus
compatibles)
   trier les types par tension décroissante                 // les
ressources rares aux voyages qui en ont besoin
   POUR chaque type, sur le MÊME état partagé :
      trier ses voyages par Difficulté croissante (le plus contraint d'abord)
      POUR chaque voyage : EssayerAffecter ; SI succès ALORS Affecter
```

6. Fonction objectif (à minimiser)

```
Objectif(état) =  100 000 x nbVoyagesNonAffectés
               +  somme des (scoreMax - score) des affectations          //
déficit de score souple
               +  10 x écartType(heures travaillées des agents)          //
équité de charge
// la pénalité domine tout : un meilleur score n'est jamais obtenu en perdant
un voyage
```

7. Amélioration : ALNS avec recuit simulé

```
Améliorer(initial) :
   courant = meilleur = copie(initial) ; T = température initiale ; poids des
opérateurs = 1
   TANT QUE itérations, durée et stagnation sont sous leurs limites ET pas
d'annulation :
      d = tirage à la roulette d'un opérateur de destruction (proportionnel
aux poids)
      r = tirage à la roulette d'un opérateur de réparation
      candidat = copie(courant)
      Détruire(candidat, entre 5 % et 15 % des voyages affectés, au moins 2)
      Réparer(candidat)                                    // toujours faisable
: règles dures respectées
      SI Objectif(candidat) < Objectif(meilleur)            : meilleur =
courant = candidat ; récompense 33
      SINON SI Objectif(candidat) < Objectif(courant)      : courant =
candidat ; récompense 9
      SINON SI aléa < exp( -(Δ objectif) / T )              : courant =
candidat ; récompense 3
      SINON                                                  : récompense 0
      poids[d], poids[r] = 0,9 x poids + 0,1 x récompense   // les opérateurs
efficaces sont tirés plus souvent
      T = T x 0,998
   RETOURNER meilleur

Destruction : aléatoire | pires scores | voisinage temporel (autour d'un
voyage non affecté)
              | par ressource (vider une partie du planning d'un bus ou d'un
agent)
Réparation  : ordre chronologique | plus contraint d'abord (avec bruit) |
ordre aléatoire (avec bruit)
```

8. Finalisation et garde-fous

```
Compléter(état) :
   trier les voyages non affectés par Difficulté croissante ; EssayerAffecter
; SI succès ALORS Affecter

ScoresExacts(état) :
   POUR chaque ressource d'une affectation : retirer temporairement le voyage
de son planning,
   évaluer les règles souples dans le même contexte qu'au choix, remettre le
voyage
   Justification = « Bus #3 : 87,5 pts [B07=40, BP04=10] | Chauffeur ... |
Receveur ... »

ContrôleIndépendant(état) :                          // ne dépend pas de
l'algorithme
   POUR chaque affectation : chauffeur apte ? receveur apte ? chauffeur ≠
receveur ?
   rejouer TOUTES les règles dures pour le bus, le chauffeur et le receveur
   toute violation = un bug du moteur

Enregistrer(plan) refusé SANS rien écrire si :
   violations > 0 | rien à planifier | aucune affectation | journée
incomplète (mode « journée complète »)
```